⚔️ PART 2 — REQUIREMENT VALIDATION

We now move from:

> **“We have written the requirements.”**

to:

> **“But are those requirements actually good and correct?”**

That process is called **Requirement Validation**.

1️⃣ WHAT IS REQUIREMENT VALIDATION?

Requirement Validation is the process of checking whether the requirements defined for the system are:

- Valid
- Correct
- Complete
- Consistent
- Feasible/implementable

The PPT explains that validation is important because **errors in requirements can cause major problems later in development**. It involves checking the requirements document and ensuring that it reflects what is actually needed.

🧠 Simple meaning

> **Requirement Validation = Checking whether we have defined the RIGHT requirements.**

2️⃣ WHY DO WE NEED REQUIREMENT VALIDATION?

Imagine this:

You build a system for months.

It is technically perfect. The code works. There are no bugs.

But then the customer says:

> “This is not what I wanted.”

😭💀

The entire project can fail because the **requirements were wrong**.

Therefore, requirement errors must be discovered **before implementation becomes expensive**.

Core idea:

> It is much cheaper to find a requirement problem early than after the system has been built.

3️⃣ VALIDATION CHECKS

The PPT gives **five major checks**.

🧠 Memory: PCCAD

P — Practically implementable
C — Correct
C — Complete
A — Unambiguous
D — Describable

Let's understand each one.

---

1. PRACTICALLY IMPLEMENTABLE

We check:

> **Can this requirement actually be implemented?**

A requirement might sound good but be impossible or impractical because of:

- Available technology
- Time
- Cost
- Resources
- Technical limitations

Example:

> “The system should never fail under any circumstances.”

That may be unrealistic/impractical.

🧠 Remember:

Implementable = Can we actually build it?

---

2. CORRECT

We check:

> **Does the requirement represent what the customer/stakeholder actually needs?**

A requirement is not correct merely because it sounds technical or well-written.

Example:

Customer wants:

> “Students should register for courses.”

But the SRS says:

> “Students can only view courses.”

The requirement is clear, but it is **not correct** because it doesn't represent the actual need.

🧠 Remember:

Correct = Is this the right requirement?

---

3. COMPLETE

We check:

> **Are all necessary requirements included?**

Nothing important should be missing.

Example:

For an online payment system:

- Payment processing ✔️
- Payment failure handling ✔️
- Refund requirements ❓

If essential requirements are missing, the requirement specification is incomplete.

🧠 Remember:

Complete = Is anything necessary missing?

---

4. UNAMBIGUOUS

A requirement should have:

> **Only one possible interpretation.**

Example:

❌ “The system should load quickly.”

What does quickly mean?

1 second?
5 seconds?
30 seconds?

✔️ Better:

> “The system shall load within 2 seconds.”

Now the requirement has a measurable, single interpretation.

🧠 Remember:

Unambiguous = One meaning.

---

5. DESCRIBABLE

The requirement should be clearly describable.

It must be expressed in a way that can be properly communicated and understood.

🧠 Remember:

Describable = Can the requirement be clearly stated and understood?

🧠 PCCAD MASTER TABLE

Validation Check	Main Question
Practically Implementable	Can we build it?
Correct	Is it the right requirement?
Complete	Is anything necessary missing?
Unambiguous	Does it have only one meaning?
Describable	Can it be clearly expressed?

4️⃣ REQUIREMENTS VALIDATION TECHNIQUES

The PPT gives **four techniques**:

1. Requirements Reviews / Inspections

2. Prototyping

3. Test-Case Generation

4. Automated Consistency Analysis

---

1. REQUIREMENTS REVIEWS / INSPECTIONS

Requirements are examined and checked.

The requirements are reviewed to detect errors, problems, and inconsistencies.

This may involve:

- Customers
- Users
- Analysts
- Developers
- Other stakeholders

🧠 Think:

> **“Let's sit down and inspect the requirements.”**

Goal:

> Find problems before development.

---

2. PROTOTYPING

A prototype is a working or visual model of part of the proposed system.

Stakeholders can interact with it and identify problems in requirements.

Example:

Before building the entire banking application, show the customer:

- Login screen
- Transfer screen
- Account dashboard

The customer may say:

> “Oh wait, I actually need an emergency account freeze option.”

That requirement was discovered through the prototype.

🧠 Prototype helps validate:

> **“Is this what you actually wanted?”**

---

3. TEST-CASE GENERATION

We derive test cases from requirements.

Then we check:

> **Can this requirement actually be tested?**

Example:

Requirement:

> “The system should be fast.”

Can we design a clear test?

❌ Difficult.

Better requirement:

> “The system shall respond within 2 seconds for 95% of requests.”

Now we can create a test case.

🧠 Remember:

Test-case generation checks whether requirements are testable.

---

4. AUTOMATED CONSISTENCY ANALYSIS

Automated tools are used to analyze requirements and identify potential:

- Inconsistencies

- Conflicts
- Problems

Example:

Requirement 1:

> The administrator can delete all user accounts.

Requirement 2:

> User accounts can never be deleted.

These conflict.

An automated analysis tool can help detect such inconsistencies.

🧠 Remember:

Automated = Tool checks for consistency/conflicts.

🔥 THE FULL REQUIREMENT VALIDATION STORY

You collect requirements:

⬇️

First ask PCCAD:

- **P:** Can we build it?
- **C:** Is it correct?
- **C:** Is it complete?
- **A:** Is it clear?
- **D:** Is it describable?

⬇️

Then use validation techniques:

- Reviews/Inspections
- Prototyping
- Test-case generation
- Automated consistency analysis

⬇️

> **Result: A validated set of requirements ready to guide development.**

🧠 THE TWO THINGS YOU MUST NOT CONFUSE

Requirement Analysis

> **Analyze and refine the gathered requirements.**

Focus:

- Understand
- Refine
- Identify issues
- Improve consistency/clarity

Requirement Validation

> **Check whether the requirements are actually acceptable and correct.**

Focus:

- PCCAD
- Validation techniques

⚡ 30-SECOND EXAM REVISION

Requirement Validation:

Checking whether the requirements are correct, complete, clear, and suitable.

Checks:

PCCAD

- Practically Implementable
- Correct

- Complete
- Unambiguous
- Describable

Techniques:

RPTA

- Requirements Reviews/Inspections
- Prototyping
- Test-case Generation
- Automated Consistency Analysis

🎯 YOUR TURN — PART 2 QUIZ

Q1.

What is Requirement Validation?

Q2.

Why is Requirement Validation necessary?

Q3.

List the **five validation checks** in the PPT.

Q4.

A requirement says:

> “The system must use technology that does not currently exist.”

Which validation check would fail, and why?

Q5.

A requirement says:

> “The application should respond quickly.”

Which validation check would you question? How would you improve the requirement?

Q6.

What is the difference between **Correctness** and **Completeness** during validation?

Q7.

Name all **four Requirement Validation Techniques**.

Q8.

A prototype is shown to the customer, and they discover that an important feature is missing. Which validation technique is being used, and how did it help?

Q9.

How does Test-Case Generation help validate requirements?

Q10.

What does Automated Consistency Analysis do?

Q11.

Differentiate between **Requirement Analysis** and **Requirement Validation** in one or two lines.

Reply in your own words, my honour 😤⚔️. After I evaluate you, we **stop**, and you decide when we move to **Part 3 — Requirements Elicitation Techniques**.

⚔️ PART 3 — REQUIREMENTS ELICITATION TECHNIQUES

We now move to **Part 3**. This part is important because the PPT gives **10 requirements elicitation techniques**. We'll cover **every technique and every sub-type** before the quiz.

🧠 First: What is Requirement Elicitation?

Requirement Elicitation = discovering and collecting requirements from stakeholders and other relevant sources.

The main question is:

> **“What does the customer and other stakeholders actually need?”**

Think of it like being a **software detective** 🕵️:

- Talk to people
- Study documents
- Observe work
- Ask questions
- Analyze interfaces
- Build prototypes
- Conduct workshops/surveys

The PPT covers the following techniques:

1. Stakeholder Analysis
2. Brainstorming
3. Interview
4. Document Analysis/Review
5. Focus Group
6. Interface Analysis
7. Observation
8. Prototyping
9. JAD / Requirement Workshops
10. Survey/Questionnaire

1️⃣ STAKEHOLDER ANALYSIS

Before gathering requirements, we need to know:

> **Who are the stakeholders?**

Stakeholders may include people such as:

- Customers
- End users
- Project team members
- Suppliers
- Other people affected by the project

The goal is to understand who has an interest in or is affected by the system.

🧠 Memory:

> **Stakeholder Analysis = WHO should we talk to?**

2️⃣ BRAINSTORMING

Brainstorming is used to generate:

- New ideas
- Solutions
- Requirements

It usually involves relevant people such as **domain experts and subject matter experts**.

According to the PPT, brainstorming can help consider:

- Expectations
- Risks
- Business rules
- Options
- Possible future problems and their prevention

🧠 Memory:

> **Brainstorming = MANY MINDS → MANY IDEAS**

3️⃣ INTERVIEW

An interview is a direct way of collecting requirements by communicating with stakeholders.

The PPT gives **two types**:

A. Structured Interview

- Questions are prepared beforehand.
- The interview follows a predefined structure.

Example:

> Q1. What are the main functions of the system?
> Q2. Who will use it?
> Q3. What problems exist in the current system?

B. Unstructured Interview

- There is no rigid predefined question structure.
- The discussion can flow freely.

🧠 Difference:

Structured	Unstructured
Predefined questions	No fixed questions
Fixed format	Flexible discussion

4️⃣ DOCUMENT ANALYSIS / REVIEW

Existing documents are studied to understand the system and its requirements.

The PPT includes documents such as:

- Business plans
- Technical documents
- Problem reports
- Existing requirements documents

This technique is especially useful for:

- Updating existing systems
- Migration projects

🧠 Memory:

Document Analysis = “Read the evidence.”

5️⃣ FOCUS GROUP

A focus group involves **subject matter experts** discussing a particular topic.

The discussion is conducted under a:

Moderator

The moderator guides the discussion, and the findings are analyzed along with business analysts.

🧠 Memory:

Focus Group = Group discussion + Moderator

Don't confuse this with brainstorming:

- **Brainstorming** → Mainly generate ideas
- **Focus Group** → Structured group discussion on a topic under a moderator

---

6 INTERFACE ANALYSIS

Interface Analysis examines how information is exchanged between:

- Systems
- People
- Processes
- Components

The PPT emphasizes these questions:

> **Who? What data? When? How? Why?**

🧠 Example

For a payment system:

- **Who?** Customer and bank
- **What data?** Payment information
- **When?** When the customer confirms payment
- **How?** Through a payment API
- **Why?** To complete the transaction

🧠 Memory:

> **Interface Analysis = How does information cross the boundary?**

---

7 OBSERVATION

In this technique, the analyst observes people while they perform their work.

This is useful when people:

- Find it difficult to explain their work
- Perform activities automatically from experience
- Forget to mention certain real-world steps

The PPT gives **two types**:

A. Active Observation

The analyst:

- Observes
- Asks questions
- May attempt the work

B. Passive Observation

The analyst:

- Silently observes
- Does not actively interfere

🧠 Memory:

> **Active = Observe + Participate/Ask**
> **Passive = Just Watch**

8️⃣ PROTOTYPING

A prototype is a model or partial representation of the proposed system.

The PPT describes using prototypes and **frequent demonstrations** to identify:

- Missing requirements
- Unspecified requirements

🧠 Why is this useful?

Sometimes a stakeholder cannot clearly explain what they want.

But when they see something:

> “Oh, that's not what I meant!”

or:

> “Wait, we also need this feature!”

💥 A requirement is discovered.

🧠 **Memory:**

> **Prototype = Show it to know what's missing.**

9️⃣ JOINT APPLICATION DEVELOPMENT (JAD) / REQUIREMENT WORKSHOPS

These are formal and structured meetings involving stakeholders/end users to:

- Define requirements
- Clarify requirements
- Complete requirements

The PPT mentions two types:

A. Formal Workshops

Structured meetings for discussing and defining requirements.

B. Business Process Improvement Workshops

Workshops focusing on improving existing business processes.

🧠 **Memory:**

> **JAD = Everyone joins together to define requirements.**

🔟 SURVEY / QUESTIONNAIRE

A survey/questionnaire is used to collect responses from stakeholders.

The PPT gives two types:

A. Open-ended Questions

The respondent gives their own answer.

Example:

> “What improvements would you like in the system?”

B. Closed-ended Questions

The respondent selects from predefined answers.

Example:

> “Do you want online payment?
> Yes / No”

🧠 Memory:

> **Open = Write your own answer**
> **Closed = Choose from given answers**

🔥 THE 10 TECHNIQUES — ONE MASTER STORY

Imagine you're building a **college management system**:

1. Stakeholder Analysis

Who should we talk to?

→ Students, faculty, management, parents, admin staff.

2. Brainstorming

What ideas and possible requirements exist?

→ Bring experts together and generate ideas.

3. Interview

Ask individuals directly.

4. Document Analysis

Read existing policies, forms, reports, and documents.

5. Focus Group

Gather experts for a moderated discussion.

6. Interface Analysis

Study how systems/people/processes exchange information.

7. Observation

Watch how people currently perform their work.

8. Prototyping

Build a sample version and show it.

9. JAD/Workshops

Conduct structured meetings to define and clarify requirements.

10. Survey/Questionnaire

Collect answers from many stakeholders.

⚠️ IMPORTANT: DON'T CONFUSE THESE

Interview vs Survey

- **Interview:** Direct conversation
- **Survey:** Questions sent to multiple respondents

Focus Group vs Brainstorming

- **Focus Group:** Moderated group discussion
- **Brainstorming:** Idea generation

Observation vs Interface Analysis

- **Observation:** Watch people doing work
- **Interface Analysis:** Study information exchange

Prototype vs Validation

- **Prototype in Elicitation:** Helps **discover missing/unspecified requirements**
- **Prototype in Validation:** Helps check whether requirements match what is actually needed

Same tool, different purpose depending on the phase.

🧠 ONE-LINE MASTER REVISION

Technique	Main purpose
Stakeholder Analysis	Identify stakeholders
Brainstorming	Generate ideas
Interview	Directly ask stakeholders
Document Analysis	Study existing documents
Focus Group	Moderated expert discussion
Interface Analysis	Analyze information exchange
Observation	Watch actual work
Prototyping	Discover missing requirements
JAD/Workshops	Jointly define and clarify requirements
Survey/Questionnaire	Collect responses

🎯 PART 3 QUIZ

Q1.

What is Requirements Elicitation?

Q2.

List all **10 Requirements Elicitation Techniques**.

Q3.

What is Stakeholder Analysis, and name any four possible stakeholders.

Q4.

What is the purpose of Brainstorming? Name any four things considered during it according to the PPT.

Q5.

Differentiate between a Structured and Unstructured Interview.

Q6.

What is Document Analysis/Review? When is it especially useful?

Q7.

What is a Focus Group, and what is the role of the moderator?

Q8.

What does Interface Analysis study? List the five key questions from the PPT.

Q9.

Differentiate between Active and Passive Observation.

Q10.

How does Prototyping help in Requirement Elicitation?

Q11.

What is JAD/Requirement Workshop? Name its two types.

Q12.

Differentiate between Open-ended and Closed-ended Survey/Questionnaire questions.

Q13. Scenario:

Match the technique:

a. Analyst silently watches employees perform their daily tasks.
b. Experts gather to freely generate ideas.
c. Existing problem reports and business documents are studied.
d. A sample application is repeatedly demonstrated to users.
e. Users answer Yes/No questions online.

Q14. The classic exam question:

Differentiate Brainstorming, Focus Group, and JAD/Requirement Workshop.

Answer when ready 😤🔥

After I evaluate, we stop again. **Next after that will be Part 4 — System Modeling.**

⚔️ PART 4 — SYSTEM MODELING

You said nextt, so we move on 😤

⚠️ Note: You skipped answering the Part 3 quiz, so I'm treating it as a self-revision/reading pass, not an evaluated completion.

This part follows your PPT and covers:

- **Meaning of System Modeling**
- **UML**
- **Benefits of System Modeling**
- **Use-Case Model**
- **Use-Case components**
- **Use-Case symbols and relationships**
- **`«extend»` and `«include»` relationships**
- **Class Diagram**
- **Visibility symbols**
- **UML relationships**
- **UML diagram types**

1️⃣ WHAT IS SYSTEM MODELING?

System Modeling means creating an abstract representation or model of a system to understand its structure and behavior.

Instead of immediately building the real system, we create models that help us understand:

- **What the system does**
- **Who interacts with it**
- **How different components relate**
- **How information or activities flow**

The PPT uses UML — Unified Modeling Language for this purpose.

🧠 Think of it like a building

You don't start constructing a building without a plan.

You first create a:

> **Blueprint**

Similarly:

> **System Model = Blueprint/representation of the software system**

2️⃣ UML — UNIFIED MODELING LANGUAGE

UML is a standardized modeling language used to:

- **Visualize**
- **Specify**
- **Construct**
- **Document**

software systems.

Different UML diagrams show different aspects of the system.

For example:

- **Use Case Diagram → Who uses the system and what they do**
- **Class Diagram → Structure of the system**
- **Sequence Diagram → Interaction order**
- **Activity Diagram → Workflow**

3️⃣ BENEFITS OF SYSTEM MODELING

The PPT lists six major benefits.

1. Manage complexity

A system may be huge and difficult to understand.

Models simplify it.

> **Model = Simplified view of a complex system**

---

2. Enhance communication

Different stakeholders can understand and discuss the system more easily through diagrams.

Instead of saying:

> **“Let me explain the entire interaction flow in 10 paragraphs…”**

You can show the model 😭.

---

3. Validate design

Models help check whether the proposed design is correct before implementation.

---

4. Document the system

Models act as documentation that can be referred to later.

---

5. Guide implementation

Models help developers understand what they need to build.

---

6. Facilitate maintenance

When the system is modified later, models help engineers understand the existing system.

🧠 **Memory:**

M E V D G F

Manage complexity
Enhance communication
Validate design
Document
Guide implementation
Facilitate maintenance

4️⃣ USE-CASE MODEL

A Use-Case Model represents the functional requirements of a system from the user's perspective.

It helps identify:

> **WHO interacts with the system and WHAT they can do.**

The main components are:

1. **Actor**
2. **Use Case**
3. **Association**

5️⃣ ACTOR

An Actor represents an entity outside the system that interacts with it.

An actor can be:

- **A user**
- **Another system**
- **An external entity**

Example: ATM System

Possible actors:

- **Customer**
- **Bank System**

🧠 Important:

An actor is not necessarily a person.

It can be another external system.

6️⃣ USE CASE

A Use Case represents a function or service that the system provides.

Example:

For an ATM:

- **Withdraw Cash**
- **Deposit Money**
- **Check Balance**

For an e-commerce application:

- **Register**
- **Login**
- **Add to Cart**
- **Make Payment**

🧠 Remember:

> **Use Case = WHAT the system does for an actor**

7️⃣ ASSOCIATION

An Association is the connection between an:

> **Actor ↔ Use Case**

It shows that the actor participates in or interacts with that use case.

Example:

Customer ———————— Withdraw Cash

---

8 USE-CASE SYMBOLS

System Boundary

A rectangle representing the boundary of the system.

Everything inside is part of the system.

Actors are generally outside.

Use Case

Usually represented using an:

Oval

The name of the system function is written inside it.

Actor

Usually represented using a:

Stick figure

Association

Usually represented by a:

Solid line

connecting an actor and a use case.

---

9 USE-CASE RELATIONSHIPS

The PPT covers two relationships:

A. «extend»

An «extend» relationship means:

> **One use case adds optional or conditional behavior to another use case.**

Example:

Withdraw Cash

▲

| «extend»

|

Print Receipt

A receipt may be printed only when required.

So the extra behavior is optional/conditional.

B. «include»

An «include» relationship represents behavior that is included as part of another use case.

Example:

Withdraw Cash

|

▼ «include»

Authenticate User

Authentication is a required/common behavior used during withdrawal.

🧠 **Memory:**

Include = MUST/always shared behavior
Extend = OPTIONAL/conditional extra behavior

🔥 NEVER CONFUSE INCLUDE AND EXTEND

Include	Extend
Required/shared behavior	Optional/conditional behavior
Base use case uses included behavior	Extra behavior extends base use case
Common functionality	Additional functionality

🔟 CLASS DIAGRAM

A Class Diagram represents the:

- **Static structure of a system**
- **Classes**
- **Attributes**
- **Operations/methods**
- **Relationships between classes**

A class is typically represented using a rectangle divided into three sections.

┌──────────────────────────┐
│ **Class Name** │ ← **Upper section**
├──────────────────────────┤

│ Attributes │ ← Middle section

├──────────────────────────┤

│ Operations │ ← Lower section

└──────────────────────────┘

1️⃣1️⃣ THREE SECTIONS OF A CLASS

Upper Section — Class Name

Contains the class name.

Example:

Student

Middle Section — Attributes

Attributes represent the data/state of an object.

Example:

Student

----

name

rollNo

email

Lower Section — Methods/Operations

Methods represent the behavior/functions of the class.

Example:

Student

---

name

rollNo

---

register()

login()

🧠 **Memory:**

Name → Data → Behavior

1️⃣2️⃣ VISIBILITY SYMBOLS

The PPT uses the following UML visibility symbols:

Symbol	Meaning
+	Public
-	Private
#	Protected
~	Package

Example:

Student

- rollNo : int

+ name : String

+ register()

1️⃣3️⃣ CLASS RELATIONSHIPS

Now comes the important section 😤.

The PPT covers:

1. **Dependency**
2. **Generalization**
3. **Association**
4. **Multiplicity**
5. **Aggregation**
6. **Composition**

A. DEPENDENCY

A dependency exists when one element uses or depends on another element.

A change in one may affect the other.

Memory:

> **Dependency = “I use you.”**

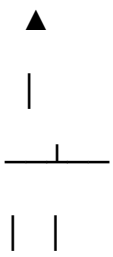
B. GENERALIZATION

Generalization represents an:

> **IS-A relationship**

Example:

Vehicle



Car Bike

A:

Car IS A Vehicle

Bike IS A Vehicle

The parent/general class provides common characteristics to subclasses.

C. ASSOCIATION

Association represents a relationship between two classes.

Example:

Student ———— Course

A student can be related to a course.

D. MULTIPLICITY

Multiplicity tells us:

How many objects of one class can be associated with another class.

Examples:

Multiplicity	Meaning
1	Exactly one

`0..1`	Zero or one
`*`	Many
`1..*`	One or more

Example:

Student ———— Course

* *

Many students can be associated with many courses.

E. AGGREGATION

Aggregation represents a:

> Whole–part relationship

But the part can exist independently of the whole.

Example:

> Department ◇——— Student

A student can still exist even if the department relationship is removed.

🧠 Memory:

> Aggregation = Has-A, but the part can live independently.

Symbol:

> Hollow diamond

F. COMPOSITION

Composition is a stronger form of whole–part relationship.

The part's existence depends on the whole.

Example:

House ◆——— Room

If the whole is destroyed, its components are conceptually destroyed with it.

🧠 Memory:

Composition = Strong ownership.

Symbol:

Filled diamond

🔥 AGGREGATION VS COMPOSITION

This is a common exam question.

Aggregation	Composition
Weak whole–part relationship	Strong whole–part relationship
Part can exist independently	Part depends on whole
Hollow diamond	Filled diamond

1️⃣4️⃣ UML DIAGRAM TYPES

The PPT mentions:

1. Activity Diagram

Shows:

> **Workflow/activities in a process**

---

2. Use Case Diagram

Shows:

> **Actors and system functionality**

---

3. Sequence Diagram

Shows:

> **Interaction between objects in time order**

---

4. Class Diagram

Shows:

> **Static structure of classes and relationships**

---

5. State Diagram

Shows:

> **Different states of an object and transitions between states**

🧠 **Memory:**

Diagram	Main question
Activity	What is the workflow?
Use Case	Who does what?
Sequence	Who interacts in what order?
Class	What is the structure?
State	What states can it be in?

🔥 PART 4 MASTER SUMMARY

System Modeling

Creates abstract representations of a system.

Benefits

MEVDGF

- **Manage complexity**
- **Enhance communication**
- **Validate design**
- **Document system**
- **Guide implementation**
- **Facilitate maintenance**

Use Case Model

WHO + WHAT

- **Actor**
- **Use Case**
- **Association**

Relationships

- `«include»` **= required/shared behavior**
- `«extend»` **= optional/conditional behavior**

Class Diagram

Name → Attributes → Operations

Visibility

+ Public
- Private
Protected
~ Package

Class Relationships

- **Dependency = uses**
- **Generalization = IS-A**
- **Association = related to**
- **Multiplicity = how many**
- **Aggregation = independent part**
- **Composition = dependent part**

Diagram Types

- **Activity**
- **Use Case**
- **Sequence**
- **Class**
- **State**

🎯 PART 4 QUIZ

Q1.

What is System Modeling?

Q2.

What are the six benefits of System Modeling?

Q3.

What is UML used for?

Q4.

What are the three main components of a Use-Case Model?

Q5.

What is an Actor? Can an Actor be another system?

Q6.

Differentiate `«include»` and `«extend»`.

Q7.

What does a System Boundary represent in a Use-Case Diagram?

Q8.

Draw/describe the three sections of a Class.

Q9.

What do these visibility symbols mean?

`+`, `-`, `#`, `~`

Q10.

Differentiate:

- Dependency
- Generalization
- Association

Q11.

What is Multiplicity? What do `1`, `0..1`, `*`, and `1..*` mean?

Q12.

Differentiate Aggregation and Composition.

Q13.

Match the diagram:

a. Shows workflow
b. Shows actors and functions
c. Shows time-ordered interactions
d. Shows static class structure
e. Shows states and transitions

Answer whenever you're ready 😤. After evaluation, the only remaining section in your CAT 2 plan will be Part 5: Requirements Management + Agile.

⚔️ PART 5 — REQUIREMENTS MANAGEMENT + AGILE

This is the final section of our CAT 2 plan.

⚠️ Important source note: Your PPT covers general Software Requirements Management. It does not appear to provide a separate detailed section specifically titled "Requirements Management in Agile." So I'll first teach exactly what the PPT covers, then clearly add the Agile connection as syllabus context.

---

1 WHAT IS REQUIREMENTS MANAGEMENT?

Requirements do not simply stay fixed forever.

During software development:

- **Customers may request changes.**
- **New requirements may appear.**
- **Existing requirements may be modified.**
- **Some requirements may become unnecessary.**

Therefore, requirements need to be managed throughout the software development process.

The PPT describes a requirements-management process involving collection, analysis, definition, prioritization, approval, tracing, implementation feedback, testing, change-impact assessment, revision, and documentation of changes.

---

2 THE REQUIREMENTS MANAGEMENT PROCESS

Your PPT's flow can be understood as:

1. Collect initial requirements

First, the initial requirements are gathered.

---

2. Analyze the requirements

The collected requirements are studied and analyzed.

---

3. Define and record the requirements

The requirements are clearly defined and recorded.

---

4. Prioritize the requirements

Requirements are prioritized.

This helps identify which requirements should receive greater attention.

---

5. Approve the requirements

The requirements are approved before proceeding further.

---

6. Trace requirements to work items

Requirements are connected to the relevant development work.

🧠 **This provides:**

> **Requirement → Related work**

---

7. Query stakeholders after implementation

After implementation, stakeholders are involved to provide feedback.

---

8. Test Management

Requirements are connected with testing activities.

The PPT later emphasizes that requirements can be linked to test cases.

---

9. Assess change impact

When a requirement changes, we ask:

> **“What else will this change affect?”**

The impact of changes is assessed.

🔟 Revise the requirements

Based on changes and analysis, requirements are revised.

1️⃣1️⃣ Document the changes

Finally, the changes made to requirements are documented.

🧠 Master flow

> **Collect → Analyze → Define & Record → Prioritize → Approve → Trace → Implement/Feedback → Test → Assess Change → Revise → Document Changes**

3️⃣ WHY IS REQUIREMENTS MANAGEMENT IMPORTANT?

Your PPT lists the following benefits:

1. Lower lifecycle cost

Better management of requirements can reduce costs throughout the software lifecycle.

2. Fewer defects

Managing requirements properly can reduce requirement-related errors.

3. Reduced risk

Understanding and controlling changes helps reduce project risk.

4. Faster delivery

Well-managed requirements help development proceed more effectively.

5. Reusability

Requirements and related information can potentially be reused.

6. Traceability

Requirements can be traced through the development process.

7. Requirements tied to test cases

Requirements can be linked with relevant tests.

8. Global configuration management

Requirements and their related changes can be managed in a controlled manner.

🧠 THE BIGGEST CONCEPT — TRACEABILITY

Remember our earlier SRS topic?

Requirements Management continues that idea.

You should be able to track:

REQUIREMENT

↓

DESIGN / WORK ITEM

↓

IMPLEMENTATION

↓

TEST CASE

So if a requirement changes, you can investigate:

- **Which work items are affected?**
- **Which implementation is affected?**
- **Which test cases need changes?**

This is why traceability is extremely useful in managing requirements.

4️⃣ CHANGE IMPACT ANALYSIS

Suppose:

> **Requirement R15 changes.**

You cannot simply change the sentence in the document and say:

> **“Done bro.” 😭**

You must ask:

- **What components depend on this requirement?**
- **Which work items are affected?**
- **Which implemented features are affected?**
- **Which test cases are affected?**

This is:

> **Change Impact Assessment**

And after assessment:

> **Revise the requirements → Document the changes**

5️⃣ REQUIREMENTS MANAGEMENT IN AGILE

⚠️ Clear distinction

The following is Agile context for your syllabus topic. Your PPT's visible Requirements Management section does not separately explain these Agile concepts.

In Agile, requirements are managed in a more iterative and change-friendly way than a one-time fixed list.

The basic idea is:

> **Requirements are continuously prioritized, selected, implemented, reviewed, and refined as development progresses.**

A simplified Agile cycle looks like:

Requirements

↓

Prioritize

↓

Select work

↓

Implement

↓

Test

↓

Get feedback

↓

Refine / Change requirements

↺

This connects directly with the PPT concepts:

- **Prioritize**
- **Trace**
- **Query stakeholders**
- **Assess change impact**
- **Revise**
- **Document changes**

So, for your syllabus, think:

General Requirements Management + Agile's continuous iteration and change

---

🔥 THE WHOLE MODULE CONNECTION

Look at the complete story:

Requirements Elicitation

> **What does everyone need?**

⬇️

Requirements Specification

> **Document it.**

⬇️

Requirements Validation

> **Is the documented requirement correct?**

⬇️

Requirements Management

> **Control and manage it throughout development.**

⬇️

Agile

> **Continuously prioritize, implement, get feedback, and adapt.**

🧠 FINAL MASTER MEMORY

Requirements Management Process

C A D P A T Q T A R D

Don't force-memorize this abbreviation right now 😭 — just understand the flow:

> **Collect → Analyze → Define → Prioritize → Approve → Trace → Query → Test → Assess → Revise → Document**

🎯 PART 5 QUIZ

Q1. What is Requirements Management?

Q2. Why is Requirements Management necessary?

Q3. Write the Requirements Management process in order.

Q4. What does it mean to prioritize requirements?

Q5. What does trace requirements to work items mean?

Q6. What is Change Impact Assessment?

Q7. What happens after a requirement change is assessed?

Q8. Name the benefits of Requirements Management mentioned in the PPT.

Q9. Explain why requirements should be tied to test cases.

Q10. Give me the complete story:

> **A customer changes one requirement during development. What should happen next according to Requirements Management?**

Q11. What is the main Agile connection to Requirements Management?

🏁 WE HAVE NOW COVERED THE ENTIRE CAT 2 PORTION

✅ **Requirements Specification + SRS**

✅ **Requirement Validation**

✅ **Requirements Elicitation Techniques**

✅ **System Modeling**

✅ **Requirements Management + Agile connection**

Now answer the Part 5 questions whenever you're ready. After that, I can evaluate your full CAT 2 readiness and give you a final exam-focused revision plan.